Prefix Sum

Introduction

- **Prefix sum**: Prefix sum is sequence of partial sum of given array. For each position of the array, the prefix sum is up to that position is the sum of all the elements form the beginning of the array up to that position.
- **Purpose of the presentation:**
  - Explain the implementation of sequential and parallel prefix sum algorithms.
  - Analysis of the time complexity, number of steps and number of operation for each of these algorithms.

Algorithms Covered

- Sequential Prefix Sum
  - Linear algorithm
- Blelloch Scan Algorithm
  - Tree based parallel algorithm with logarithmic depth
- Work-Efficient Prefix Sum
  - Chunk based parallel algorithm

Sequential Prefix Sum

- Iterating through each element of the form the index 1.
- And adding the value of previous index to the current index and saving the sum in the current index.
  - prefixSum[i] = prefixSum[i - 1] + arr[i];

Sequential Prefix Sum Analysis

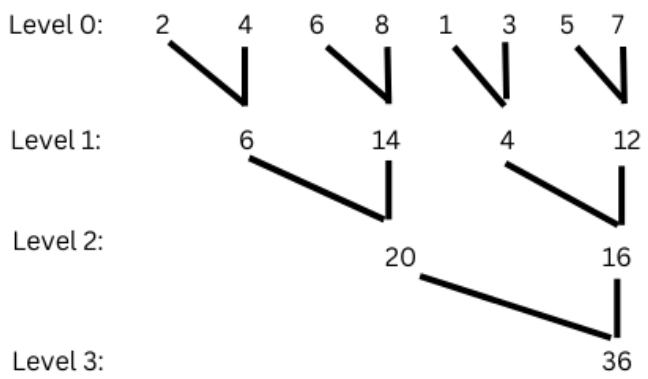
- Assuming number of input is n.
- **Time Complexity**: O(n)
  - As we are processing each element once.
- **Number of steps**: n – 1
  - We go through the whole array from index 1.
- **Number of operation:** n – 1
  - We don’t do the addition on the 0^{th} index. We start to do it from index 1.

Blelloch Scan Algorithm Overview

- **Upsweep Phase:**
  - Build a sum tree by adding pairs of elements
  - At each level, we get partial sums of two adjacent nodes of the tree
  - Parallel computation across the element
- **Setting Last Element To Zero and save the value in a variable:**
  - Prepares for exclusive sum
- **Downsweep Phase:**
  - This traverse the upsweep tree backwards.
  - That is in each level we are still working with two inputs but unlike upsweep where we were getting one output, here we'll get two outputs when operating on two nodes.
  - Left output is simply right node copied to the left and right output is sum of left node and right node.
  - And to do that we will use the partial sum that we got from the upsweep.

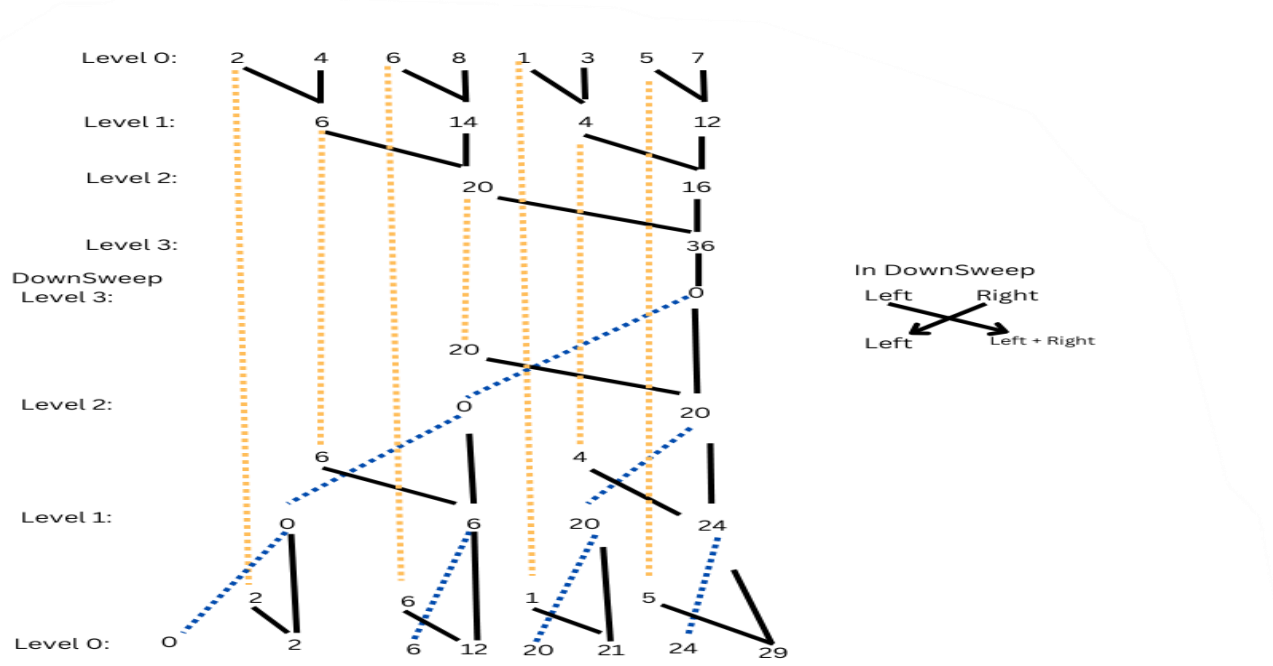
Blelloch Scan Algorithm Explanation

- Upsweep phase with input array {2, 4 , 6, 8, 1, 3, 5, 7}



Blelloch Scan Algorithm Explanation

- Setting the last element 36 to 0, which would become the root node for the downsweep.
- **Downsweep Phase:**



Blelloch Scan Prefix Sum Analysis

- Assuming number of input is n.
- **Time Complexity**: O(log n)
  - Log n levels on both phases.
- **Number of steps**: 2 * log n
  - Upsweep log(n) steps and downsweep log(n) steps.
- **Number of operation: 2(**n – 1)
  - Upsweep n – 1 addition
  - Downsweep n – 1 addition

Work-Efficient Prefix Sum Algorithm Overview

- Divides array into chunks.
- Each thread calculates partial prefix sum on its chunks.
- For each chunk, replace the second element of the chunk with the partial sum.
- Also stores the partial sums.
- Calculates offset. That means each thread cumulative sum of all the preceding partial sums as its offset.
- Each thread applies the offset to its chunks. And combining values from all thread we get the final prefix sum.

Chunk Based Prefix Sum Explanation

- For the Given input:
- We first calculate the chunk size: {2, 4, 6, 8, 1, 3, 5, 7}

chunkSize = (sizeOfTheArray + NumberOfThread - 1) / NumberOfThread

= (8 + 4 – 1) / 4 (4 is the common default for parallel programming)

= 11 / 4

= 2 (we take the integer value)

Step 1:

- Now on each thread we will compute the prefix sum of the chunks:
  - Thread 0 : chunk {2, 4} → {2, 2 + 4} → {2, 6}
  - Thread 1: chunk {6, 8} → {6, 6 + 8} → {6, 14}
  - Thread 2: chunk {1, 3} → {1, 1 + 3} → {1, 4}
  - Thread 3: chunk {5, 7} → {5, 5 + 7} → {5, 12}

Step 2:

- Store the partial sum: partialSum[0] = 6 , partialSum[1] = 14, partialSum[2] = 4, partialSum[3] = 12

Chunk Based Prefix Sum Explanation

Step 3:

- To calculate offset, each thread calculates the cumulative sum of all the preceding partial sum as its offset.
  - Thread 0: Offset = 0 (as there is no preceding partial sums)
  - Thread 1: Offset = partialSum[0] = 6
  - Thread 2: Offset = partialSum[0] + partialSum[1] = 6 + 14 = 20
  - Thread 3: Offset = partialSum[0] + partialSum[1] + partialSum[2]
    = 6 + 14 + 4
    = 24

Step 4:

- Each thread applies offset the its chunk.
  - Thread 0: Offset = 0, chunk= {2, 6}, result = {2 + 0, 6 + 0} = {2, 6}
  - Thread 1: Offset = 6, chunk= {6, 14}, result = {6 + 6, 14 + 6} = {12, 20}
  - Thread 2: Offset = 20, chunk= {1, 4}, result = {1 + 20, 4 + 20} = {21, 24}
  - Thread 3: Offset = 24, chunk= {5, 12}, result = {5 + 24, 12+ 24} = {29, 36}
- Combining all the chunks from all the threads, we get our prefix sum {2, 6 12, 20, 21, 24, 29, 36}

Chunk Based Prefix Sum Analysis

- Assuming number of input is n and p is number of threads.
- **Time Complexity**: O(n / p + p)
  - Partial sum in chunks: O (n / p)
  - Offset calculation: O(p)
- **Number of steps**: O(n / p + p)
- **Number of operation: 2(**n – 1)
  - For Chunks Partial sum: n – p addition
  - For offset calculation: (p(p – 1)) / 2 addition
  - For offset application: n addition
  - Total: O (n + p^2) addition

Comparison of all three Algorithms

Algorithm	Time Complexity	Steps	Operations
Sequential Prefix Sum	O(n)	n−1	n−1 additions
Blelloch Scan Algorithm	O(logn)	2 logn	2n−2 additions
Chunk Based Parallel Prefix Sum	O(n/p+p)	O(n/p+p)	O(n+p^2) additions